MULTILINGUAL MULTIMODAL TOURISM AGENT FOR SRI LANKA USING KNOWLEDGE GRAPH REASONING

Project Id: 25-26J-119

Project Proposal Report

Palliyaguru M D

B.Sc. (Hons) Degree in Information Technology Specialized in Data Science

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

MULTILINGUAL MULTIMODAL TOURISM AGENT FOR SRI LANKA USING KNOWLEDGE GRAPH REASONING

Project Id: 25-26J-119

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology Specialized in Data Science

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Palliyaguru M D	IT22066466	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision

Signature of the supervisor: Date: 29/08/23

ABSTRACT

This proposal outlines a research plan to develop an integrated multi-agent orchestration framework for an intelligent tourism system in Sri Lanka. The system addresses a key gap in current travel planning by enabling context-aware coordination among specialized AI services through a knowledge-graph-first approach and multi-protocol communication. Unlike existing travel assistants that depend on a single large language model or rigid rule-based methods, this solution unifies four complementary components into one architecture.

The first component, a Multilingual & Multimodal Language Service, will support Sinhala–English code-mixed speech recognition, text-to-speech, and Sinhala OCR, ensuring inclusive interaction. The second, a Tourism Knowledge Graph and Reasoning Service, will serve as contextual memory, ingesting official datasets and enabling explainable semantic and geo-spatial reasoning. The third, a Recommendation & Sustainability Service, will provide personalized and eco-conscious suggestions using forecasting, micro-moment triggers, and sustainability scoring. Finally, a Multi-Agent Orchestrator will parse user intents, generate task graphs, and coordinate domain-specific agents (Hotels, Transport, Visa/Policy, Attractions) to collaboratively plan itineraries and manage transactions.

To support collaboration, the design integrates communication protocols—including a Model Context Protocol (MCP) adapter for tool-like calls, an Agent-to-Agent (A2A) messaging interface, and a Blackboard shared memory mechanism—allowing performance comparison. Event-driven adaptability, such as weather alerts and crowd updates, and sustainability guardrails further enhance responsible tourism practices. Implementation will proceed in phases: language and knowledge graph services, orchestration core, and recommendation triggers. Evaluation will emphasize cultural relevance, explainability, sustainability impact, and task success.

By addressing the gap in contextual multi-agent orchestration, this project proposes a novel architecture that strengthens itinerary planning and advances knowledge-driven agent coordination in real-world AI systems.

Keywords: Multi-agent Systems, Knowledge Graph, Tourism IT, Agent Communication Protocols, Context-aware AI, Sustainable Tourism, Orchestration Framework, Sri Lanka.

Table of Contents

DECLARATION i
ABSTRACT ii
LIST OF FIGURES iv
LIST OF TABLES v
LIST OF ABBREVIATIONS vi
1.INTRODUCTION 1
1.1 Background and Literature Review 3
1.2 Research Gap 10
1.2 Research Problem 15
2. OBJECTIVES 17
2.1 Main Objective 17
2.2 Specific Objectives 17
3. METHODOLOGY 19
4.PROJECT REQUIREMENTS 23
4.1 Functional Requirements 23
4.2 Non-Functional Requirements 25
4.3 System Requirements 27
4.4 User Requirements 27
5.GANTT CHART 28
5.1 Work Breakdown Structure 28
6.PERSONAL AND FACILITIES 29
7. BUDGET AND BUDGET JUSTIFICATION 29
7. REFERENCES 31

LIST OF FIGURES

Figure 3.1Overall system diagram 19
Figure 3.2Component Diagram.....20
Figure 5.1:Gantt Chart 28
Figure 5.2:Work Breakdown Structure 28

LIST OF TABLES

Table 1.1: Existing Research Comparison 14
Table 7.1Budget Table 30

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
KG	Knowledge Graph
LLM	Large Language Model
MCP	Model Context Protocol
A2A	Agent-to-Agent Protocol
ACP	Agent Communication Protocol
ANP	Agent Network Protocol

1.INTRODUCTION

Tourism is a complex domain that involves planning and coordinating multiple services – from transport and accommodation to attractions, events, and adherence to local policies. Travelers increasingly expect personalized, real-time itinerary planning that can adapt to changing conditions and preferences [2]. In Sri Lanka, a country rich with cultural nuances, seasonal patterns, and sustainability concerns, an intelligent tourism assistant must handle not only generic travel information but also context-specific knowledge (e.g. public holidays like Poya days, monsoon weather patterns, heritage site regulations). Traditional online travel platforms and single-agent chatbots have limitations in this regard. Many rely on static datasets or siloed services, lacking a unified context and failing to dynamically coordinate tasks when unexpected situations arise (e.g. sudden weather disruptions or overbooking).

Recent advances in AI suggest that deploying multiple specialized agents working in unison could overcome these limitations [4]. For example, a travel planner agent could collaborate with a weather agent, transport agent, and hotel booking agent to cover all aspects of a user's trip, providing comprehensive and up-to-date recommendations [4]. However, orchestrating such a multi-agent system introduces its own challenges, particularly in ensuring the agents share relevant context, avoid conflicting actions, and adhere to common goals. Inter-agent communication protocols and a shared memory or knowledge base become crucial to enable seamless coordination. Crucially, contextual knowledge about Sri Lanka's tourism domain needs to be deeply integrated. A Knowledge Graph (KG) representing tourism entities (places, routes, events, policies) can serve as a collective memory for agents [5]. Knowledge Graphs enable AI agents to navigate complex webs of relationships and context, acting as a "cognitive backbone" that mirrors how humans understand the world. By leveraging a Sri Lanka Tourism KG, agents can ground their decisions in factual, up-to-date information about destinations, travel rules, cultural norms, and even sustainability indicators. This addresses a known weakness of

LLM-based agents: the tendency to hallucinate or miss fine-grained constraints. In tourism applications, factual accuracy and contextual suitability are paramount – the KG helps ensure that recommended itineraries are realistic and culturally appropriate. As evidenced in recent work, using a domain-specific knowledge graph dramatically improves precision and reduces errors in travel responses [6]. For instance, Song et al. (2024) demonstrate that a travel knowledge graph serving as the core retrieval source for an LLM significantly reduces erroneous recommendations and hallucinations [6].

Another frontier in this domain is the use of multiple agent communication protocols to facilitate coordination. Historically, multi-agent systems in research used approaches like Blackboard architectures (where agents post and read from a common data store) or custom message-passing schemes. Today, with the rise of LLM-based agents, new protocols are emerging to standardize how agents invoke tools and exchange context. The Model Context Protocol (MCP), for example, provides a JSON-RPC based client–server interface for secure tool invocation and structured data exchange [3]. Meanwhile, Google's Agent-to-Agent (A2A) protocol enables direct peer-to-peer task delegation using capability-based "Agent Cards" for rich interactions [3]. There are also efforts like the Agent Communication Protocol (ACP) for RESTful messaging and an Agent Network Protocol (ANP) for decentralized agent marketplaces [3]. Each protocol offers different advantages in terms of interaction patterns and interoperability. This project plans to incorporate multiple protocols (MCP, A2A, and a Blackboard-style shared context) within the same system. Doing so not only allows the system to operate in different modes but also provides a research opportunity to benchmark the protocols in a controlled domain context. As Ehtesham et al. (2025) note, standardized agent communication is essential for modular and scalable multi-agent workflows, yet clear best practices remain nascent [9],[10]. By implementing and comparing protocols in one tourism orchestration framework, we can observe their impact on task success, latency, and complexity in a real-world use case.

In summary, the proposed research seeks to fill the gap between single-agent travel planners and fully integrated multi-agent tourism systems. We aim to design a context-aware multi-agent orchestration system tailored for Sri Lankan tourism, where: (1) a central Orchestrator intelligently manages a team of specialized agents, (2) a knowledge graph provides a unified and updateable context store, (3) multiple communication protocols are available for agent interoperability, and (4) domain-specific considerations (cultural norms, sustainability, human oversight) are built into the loop. This introduction has outlined the motivation and high-level vision. The following sections will delve into the current state-of-the-art, identify the specific research gaps, formulate the research problem, and detail how we intend to achieve the objectives through a structured methodology.

1.1 Background and Literature Review

Intelligent travel planning has become a major focus as global tourism grows and AI capabilities advance [1]. Planning an itinerary involves complex, multi-dimensional constraints – from destinations and schedules to budgets and personal preferences [1]. Traditional trip planning platforms are largely static, offering generic results that lack real-time adaptation [7] . Recent approaches leverage Large Language Models (LLMs) as travel assistants to provide more personalized and dynamic plans. For example, TravelAgent (Chen et al., 2024) uses an LLM-driven system with modules for tool use, recommendation, planning, and memory to generate rational, comprehensive, personalized itineraries, outperforming rule-based methods [1]. Likewise, Singh et al. (2025) introduced Travel Optix, a multi-agent travel planner that assembles itineraries in under 5 minutes using specialized agents to gather live data (weather, events, lodging, etc.), yielding more accurate and up-to-date plans than single LLMs [9]. These systems illustrate the potential of AI in tourism but also reveal limitations: single-agent solutions or monolithic LLMs often struggle to satisfy all criteria simultaneously [1], and to handle dynamic changes or fine-grained local context.

To tackle complex tasks beyond a single agent's scope, researchers are turning to multi-agent system (MAS) architectures [10]. The rationale mirrors human teamwork – multiple specialized agents collaborating can solve problems that exceed the ability of one agent alone [10]. LLM-driven agents are particularly skilled at different subtasks (e.g. route planning, recommendation, question answering), and by working in parallel and sharing information they can cover a broader problem space. Modern AI agents incorporate memory, planning skills, tool usage and interactive abilities [10], yet even the most advanced individual agent faces limits in extended reasoning or parallel task execution [10]. Multi-agent orchestration addresses this by distributing cognitive labor: for instance, one agent can focus on high-level itinerary planning while others handle real-time data queries, transport arrangements, lodging information, or local navigation details. [10]. Such specialization and concurrent operation offer greater autonomy and effectiveness in real-world travel scenarios [10]. However, implementing effective coordination poses challenges – agents must maintain a shared context, avoid duplicating work, and integrate their results into a coherent output for the user [10].

Context management is a critical hurdle identified in prior research [10]. In multi-agent dialogues, information learned by one agent may not transfer seamlessly to others, causing discontinuities ("disconnected models' problem" [10]). Memory of past interactions can be lost as agents hand off tasks, and important details might be overlooked if there is no common knowledge store [10]. For a tourism assistant, this could mean an agent recommending an attraction that another agent already deemed unsuitable, simply because they lacked a unified context. Researchers have highlighted that autonomous agents need to "carry context through a bunch of actions" to be truly effective [10]. Recent work on the Model Context Protocol (MCP) aims to solve this by standardizing how agents share state and data [10]. MCP provides a structured way for agents to retain and exchange context, akin to a common blackboard or memory hub, and has shown promise in keeping multi-agent workflows coherent [10]. In our tourism domain, leveraging such context-sharing mechanisms (e.g. via a knowledge graph) could ensure that all domain agents

(hotels, transport, events, etc.) operate with a collective situational awareness instead of in isolation.

Multi-Agent Systems in Tourism: Several studies have applied MAS to travel planning, demonstrating improved adaptability and user satisfaction. Vaiage (Liu et al., 2025) is a recent multi-agent framework that addresses the "static itinerary" problem by introducing cooperative agents for different functions [7] . Vaiage's graph-structured MAS uses LLM-based agents both as goal-conditioned recommenders and sequential planners, allowing dynamic refinement of itineraries as conditions change [7]. The agents incorporate real-time context like budget, timing, group size, and even weather constraints, producing plans that are adaptive and explainable through natural language interactions [7]. In evaluations, Vaiage's coordinated agents (e.g. a Strategy Agent and an Information Agent) significantly improved itinerary feasibility and quality, compared to ablations without such coordination [7]. This underscores that agent coordination and specialization (one agent focusing on optimizing time use, another on integrating live data) yield more realistic travel plans [7]. Similarly, Deng et al. (2025) identify key gaps in existing travel services – intelligent multi-stop planning, last-mile navigation guidance, and dynamic re-planning – and propose a trio of agents to fill these gaps [11]. Their Travel Planning Agent uses map-based spatial reasoning to handle complex queries, a Destination Assistant Agent gives fine-grained guidance for the last 100 meters of navigation, and a Local Discovery Agent uses image-based retrieval augmentation to detect disruptions and adapt the itinerary on the fly [11]. This multi-agent setup led to measurable improvements in query interpretation and navigation accuracy in user studies [11]. The success of such domain-specific MAS solutions (TravelAgent, Travel Optix, Vaiage, Geo-Experience framework) confirms that breaking the problem into cooperating agents can enhance different aspects of the travel experience – from personalized recommendations to robustness against real-world changes [11].

MAS Frameworks and Protocols: On the systems side, there is active development of frameworks to manage inter-agent communication and coordination. Traditional multi-agent systems often relied on bespoke communication or blackboard models, but with LLM-based agents, new standardized protocols are emerging to improve interoperability [3]. Chen et al. (2025) provide a comparative survey of four prominent agent interoperability protocols – Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent (A2A), and Agent Network Protocol (ANP) [3]. Each targets a different layer of interaction. MCP(pioneered by Anthropic) defines a JSON-RPC based client–server interface for secure tool calls and structured data exchange [3]. It essentially lets an agent invoke tools or services in a standardized way, sending context (inputs, parameters) and receiving results in a consistent JSON schema. This is highly relevant for a travel orchestrator agent calling, say, a hotel booking API or a weather service – MCP would ensure the context is packaged and delivered in a uniform format [3]. ACP, by contrast, is a RESTful messaging protocol enabling multi-part asynchronous messages (with support for streaming) between agents, suited for rich multimodal content and long-form exchanges[3]. For instance, ACP could allow an image analysis agent and a text-based agent to exchange a combination of text and image data within one message envelope. A2A (introduced by Google) is a peer-to-peer agent protocol that uses capability-based Agent Cards for discovering and delegating tasks to other agents, facilitating more decentralized, scalable workflows across agents or services[3]. This could be used in a tourism context to let an itinerary-planning agent delegate a sub-task (like finding local events) to a specialized events agent, with the A2A handshake ensuring the receiving agent has the capability and agreeing on the task contract. Finally, ANP aims at open agent ecosystems – using decentralized identifiers and semantic graphs for agent discovery and trust management in a marketplace of agents [3]. While ANP is still nascent, it envisions a future where, for example, a travel agent could discover third-party mobility agents (ride-sharing bots, etc.) on the fly and securely collaborate.

These protocols are complementary, and the survey proposes a phased adoption: start with MCP for basic tool integration, layer in ACP for richer interactions, use A2A for advanced agent delegation, and eventually ANP for open networks [3]. However, most current MAS implementations use at most one protocol. Anthropic's MCP was showcased in contexts like tool former agents and knowledge management[10], and Google's A2A in enterprise task orchestration, but few systems combine them [12] . Liao et al. (2025) specifically note that "very few applications exist where both protocols are employed within a single MAS framework" [12]. They developed AgentMaster, a modular MAS that implements both A2A and MCP, demonstrating that multi-protocol coordination is feasible and advantageous [12]. AgentMaster provides a unified conversational interface through which a user can interact with the multi-agent system, while under the hood agents coordinate via whichever protocol is optimal (MCP for tool usage, A2A for agent delegation) [12]. Their evaluation reported robust task decomposition, dynamic tool routing, and high-quality results on multimodal information queries [12]. This indicates that a protocol-agnostic orchestrator can flexibly route tasks: e.g., use MCP to query a mapping service, then use A2A to get help from a peer agent, all in one workflow. Such flexibility is highly relevant to our tourism orchestrator, which should handle different modes of interaction (querying web services vs. chatting with domain agents).

Beyond low-level protocols, frameworks like AutoGen (Wu et al., 2023) provide higher-level tools for building multi-agent conversations [13]. AutoGen allows developers to define multiple LLM agents with roles and have them converse to solve tasks, using either natural language or programming constructs to script their interaction patterns [13]. This enables dynamic conversation flows rather than rigid dialogues – for example, one can create an agent team where if Agent A's answer is uncertain, Agent B steps in to fact-check using a tool, and Agent C summarizes the final output. These conversation patterns (e.g. the ReAct pattern, debate, or brainstorming) have been recognized as crucial for collaborative LLM agents [14]. Surveys of LLM-based multi-agent systems (e.g. Aratchige & Ilmini 2025) emphasize the importance of such architectures, memory

mechanisms, and planning strategies to optimize collaboration [14]. They highlight recent innovations like the Mixture-of-Agents architecture and advanced planning models, which improve how agents divide roles and make decisions [14]. At the same time, persistent challenges are noted: scalability to real-time constraints, ensuring agents' responses remain consistent and coordinated, and avoiding conflicts or redundant actions [14]. These challenges align with the needs of a live tourism assistant – it must be responsive (fast queries), coordinated (hotel agent and transport agent should work in concert), and scalable as new data or agents (e.g. a new airline API) are added.

Knowledge Graphs for Context and Reasoning: A key theme in emerging MAS research is the integration of Knowledge Graphs (KGs) to provide a structured, persistent memory and context for agents. As multi-agent tasks scale in complexity, the information they handle becomes intricate and interconnected [15]. Graph-based knowledge representations can significantly aid agents in organizing and reasoning over such information [15]. Bei et al.(2025) present the first systematic review of combining graphs with AI agents, concluding that graphs serve as powerful data structurization tools to help agents plan, remember, and cooperate more effectively [15]. In their survey, graphs are shown to empower core agent functionalities: for instance, representing the environment or world state as a graph can improve an agent's long-term memory and relational reasoning; using a KG for common sense or domain knowledge can prevent factual errors or nonsensical actions [15]. The authors argue that advanced agents will increasingly rely on graph-structured knowledge to navigate "intricate webs of relationships, dependencies, and contexts" in real-world tasks [5] . This is highly pertinent to tourism, where a traveler's query might involve relationships between places, distances, times, and personal preferences (a graph is a natural way to encode destinations, their locations, transportation links, lodging options, etc., with edges carrying semantic meaning like isLocatedIn or opensAt).

In practical terms, a knowledge graph can act as the shared blackboard for multi-agent collaboration. Nicola Rohrseitz (2025) describes KGs as the "cognitive backbone" of AI agents, giving them a shared understanding of entities and their relations, much like humans rely on connected knowledge [5]. By storing intermediate results and world facts in a graph, all agents can query and update a common source of truth. This mitigates the context fragmentation problem – rather than passing long text prompts back and forth, agents can post the facts they discover into the graph and read what others have posted. For example, a Sightseeing Agent could add "Sigiriya Rock is closed on Poya days" to the KG, and a Planning Agent later considering a Monday (which is a Poya) will see that fact and avoid Sigiriya in the itinerary. Knowledge graphs also inherently provide explainability: each node or edge can carry provenance info (which agent/tool provided this fact), enabling the system to later explain why a certain recommendation was made . This approach echoes the idea of Knowledge Graph of Thoughts (KGoT)proposed by Besta et al. (2025). KGoT dynamically builds a knowledge graph out of an LLM agent's reasoning process, structuring the task-relevant information and iteratively refining it by calling external tools [16]. Impressively, even lightweight models guided by KGoT achieved ~29% higher task success on a complex benchmark and dramatically lower costs, compared to using a powerful LLM without a KG memory [16]. This shows that structuring knowledge into a graph not only improves accuracy but also efficiency, as the model can focus on the distilled facts rather than re-processing raw text repeatedly.

Despite these advances, current literature shows certain gaps that our proposed orchestrator aims to fill. Notably, none of the reviewed travel planning systems incorporate all of the following: dynamic agent management, multi-protocol interoperability, a knowledge-graph-centric memory, deep local context awareness, and transactional capabilities with human oversight. We next outline the specific research gaps and how our approach will address them.

1.2 Research Gap

Based on the state of the art, we identify several key gaps that our Multi-Agent Orchestrator for Tourism will address:

1. **Dynamic Agent Spawning & Lifecycle Management**: Gap: Existing MAS frameworks in literature assume a fixed set of agents defined at design time. They do not support on-demand creation or retirement of agents based on user intent, nor robust failure handling (retries, timeouts, circuit-breakers) for unreliable agents. For example, Travel Optix's agents are predetermined for itinerary, data fetching, etc., and if one fails, the system has limited recourse beyond returning partial results [2].

   Our Novelty: We propose an orchestrator that can spawn specialized agents dynamically when a task requires (e.g. create a "WildlifeGuideAgent" if the user asks about wildlife parks), manage their lifecycle (ensure they complete or time out), and gracefully handle errors. This includes maintaining pools of hot agents for frequent tasks and using circuit breakers to avoid bottlenecks from failing services. Such dynamic orchestration, with agents as ephemeral yet coordinated subprocesses, is not explored in prior tourism MAS research and directly increases the system's flexibility and fault tolerance.

2. **Tourism-Domain Agent Pack with Canonical Schemas**: Gap: Prior travel assistant systems each craft their own data formats and agent APIs, leading to siloed designs. There is no standardized schema for information like an "Offer" (hotel offer, tour offer), "TripLeg" (transport segment), or "PolicyRule" (visa or COVID regulation) that all agents agree on. This lack of a canonical data model means agents from different developers or domains have trouble interoperating. Even AgentMaster's multi-protocol MAS focuses on communication layers but not on agreeing what the content means domain-wise[12].

Our Novelty: We introduce a domain-specific agent pack for tourism, where each agent (Hotel, Transport, Visa/Policy, Events, Attractions, Emergency) adheres to a minimal canonical schema for inputs/outputs. For instance, a HotelAgent always returns offers in a defined JSON structure with fields for location, price, cancellation policy, etc., and a TransportAgent produces TripLeg objects with standardized fields (origin, destination, mode, duration, cost). By using a shared ontology (aligned with the tourism KG and industry standards), our agents can plug-and-play and the orchestrator can merge results easily. This is the first attempt at a standardized schema for tourism agents, ensuring interoperability across tools and even protocols – an agent can be called via MCP or A2A, and the orchestrator knows how to interpret the result because the schema is known.

3. **KG-First Context Plane with Evidence Trails**: Gap: While knowledge graphs are recognized as useful, no current MAS in the travel domain uses a knowledge graph as the primary context memory for agent coordination. Typically, agents communicate by passing messages or long prompts, which can lead to inconsistencies and lost information over time [10]. There is also little focus on explainability – users are left to trust the AI's suggestions without insight into its reasoning or data sources.

   Our Novelty: We adopt a Knowledge-Graph-centric architecture where the orchestrator and all agents read from and write to a shared Tourism Knowledge Graph (which contains facts about destinations, user preferences, interim decisions, etc.). This KG-first approach serves as a blackboard: e.g., once the VisaAgent notes "ETA visa required for traveler's nationality" in the KG, the itinerary planner agent sees that and can adjust the plan accordingly (ensuring sufficient visa processing time or suggesting alternatives). Every piece of information in the KG is stored with provenance (which agent/tool provided it and when), enabling an evidence trail. This KG-first orchestrator is novel in combining real-time multi-agent operations with persistent, queryable memory, and it directly

addresses the context fragmentation problem highlighted by Krishnan (2025) [10]. It also meets the rising demand for explainable AI in tourism – travelers and providers can trace why a certain plan was made (important for trust and accountability).

4. **Formal Multi-Protocol Benchmarking (MCP vs. A2A vs. Blackboard)**: **Gap:** There is currently no head-to-head evaluation of different agent orchestration paradigms on a common set of tasks. Surveys propose how protocols might complement each other [3], and benchmarks like AgentBench and LiveMCPBench test agent performance in certain environments [17], but these do not compare orchestration strategies. AgentBench evaluates individual LLM agents across tasks, finding that even top models struggle with long-term reasoning and decision-making [17], while LiveMCPBench stresses agents in a rich MCP tool ecosystem (10,000+ tools) to gauge their tool-use capabilities [18]. However, we lack insights into trade-offs between a pure blackboard (central KG) approach versus a message-passing approach (A2A/ACP) versus an API-calling approach (MCP) in a unified setting.

   Our Novelty: We plan to *experimentally assess* our orchestrator under different coordination modes rather than attempting a full-scale benchmark implementation. By toggling between modes (e.g., blackboard-only, MCP calls, peer-to-peer calls) on representative tourism tasks such as "plan a 5-day cultural tour with real-time weather updates," we will *test and compare* end-to-end success rates, latency (p95 response time), robustness under partial failures, and ease of adding new agents. This exploratory evaluation will extend existing efforts such as LiveMCPBench [18] by incorporating A2A and blackboard scenarios, and will provide practical insights into trade-offs between orchestration approaches. While prior work (e.g., Liao et al. [12]) suggests that a hybrid approach may offer the best flexibility, our study aims to quantify the overheads and failure modes in practice. The

contribution will thus be one of the first attempts to produce comparative guidelines for when to prefer MCP (tool invocation), A2A (direct agent queries), or blackboard (shared KG) coordination in real-world tourism tasks.

5. **Transactional Integrations with Human-in-the-Loop**: **Gap:** Most research prototypes in travel planning stop at generating itineraries or recommendations and do not integrate with real booking systems or transaction flows, largely due to complexity and risk. This leaves a gap in showing how AI-generated plans can transition into actual actions (e.g., booking a hotel or purchasing a train ticket) in a trustworthy manner. Moreover, fully automating financial transactions introduces trust concerns—users are often reluctant to let an AI spend money or make binding reservations without oversight. Existing systems thus avoid the "final mile," or when they attempt transactions, they rarely include a human confirmation step.

   Our Novelty: Rather than building a full transactional platform, we will demonstrate *proof-of-concept integrations* with selected booking APIs (e.g., hotels, trains/buses, event tickets) to illustrate how our orchestrator could handle real reservations. All transactions will be human-in-the-loop: the orchestrator will prepare a "transaction proposal" (such as a filled cart of selected bookings) and present it to the user for explicit one-click confirmation. Every action will be logged in the knowledge graph with full provenance (which agent recommended it, which API was invoked, timestamps), supporting auditability and rollback. This approach bridges the gap between AI planning and execution while preserving user trust and control. It also enables valuable feedback loops: if a user rejects a proposed booking, that decision can be recorded in the KG (e.g., due to a missed preference) and inform future recommendations. By showcasing a working loop from intent → planning → mock transaction proposal → confirmation, our system highlights a feasible path toward operationalizing AI travel assistants in live environments—an aspect largely absent in current literature, which tends to

evaluate only planning accuracy or user satisfaction in abstract, rather than transaction readiness [12].

Research Gap / Novelty	**A**	**B**	**C**	**Our System**
1. Dynamic Agent Spawning & Lifecycle Management	✗	✗	✗	✓
2. Tourism-Domain Agent Pack with Canonical Schemas	✗	✗	✗	✓
3. KG-First Context Plane with Evidence Trails	✗	Partial (graph memory only)	✗	✓
4. Formal Multi-Protocol Benchmarking (MCP vs. A2A vs. Blackboard)	✗	✗	✓ (MCP-only)	✓
6. Transactional Integrations with Human-in-the-Loop	✗	✗	✗	✓

Table 1.1: Existing Research Comparison

A – Automated Travel Planning Via Multi-Agent Systems and Real-Time Intelligence [9]

B – Vaiage: A Multi-Agent Solution to Personalized Travel Planning [7]

C – Livemcpbench: Can agents navigate an ocean of mcp tools? [18]

The comparison table shows that none of the existing systems fully address the identified gaps. Most lack support for dynamic agent spawning, standardized tourism schemas, or a knowledge-graph-first context plane. Protocol coverage is also partial, with current efforts focusing mainly on MCP-only, and none demonstrate safe transactional integration. Our

system uniquely covers all five gaps, making it the first holistic approach to multi-agent orchestration in the tourism domain.

1.2 Research Problem

Based on the gaps identified, the research problem can be formally defined as follows:

How can we design, implement, and evaluate a knowledge-graph-driven multi-agent orchestration framework for intelligent tourism planning in Sri Lanka that enables context-aware coordination among heterogeneous agents via multiple communication protocols, to improve task success, efficiency, and cultural appropriateness of travel itineraries compared to existing single-agent or ad-hoc systems?

This problem statement can be unpacked into sub-questions:

Design Aspect: What architectural components and interactions are necessary to orchestrate domain-specific agents (for hotels, transport, etc.) under a central system, and how can a shared knowledge graph be integrated as the context backbone for these agents?

Coordination Aspect: In what ways can multiple agent communication protocols (like MCP and A2A) be utilized concurrently or adaptively within one system, and what mechanisms decide which protocol to use for a given inter-agent interaction or tool invocation?

Functional Efficacy: To what extent does the proposed multi-agent system improve the outcomes of travel planning (itinerary quality, user satisfaction, error reduction) over the baseline approaches? Specifically, can it handle complex constraints (time, budget, visa rules) more effectively by dividing tasks among specialized agents and using shared context?

Cultural and Sustainability Integration: How can rules and triggers for local cultural norms and sustainability be incorporated into the agent reasoning process without human intervention, and what impact does this have on the appropriateness and responsibility of

generated Itineraries?

Human Oversight: What is an optimal method to involve the human user in the loop for critical decision points (like final booking), and how does this interaction interface with the agent orchestration (e.g., pausing agent actions, revising knowledge base with user feedback)?

Evaluation: What metrics and experimental scenarios will best measure the system's performance on both technical metrics (like latency, fault tolerance) and domain-specific metrics (like cultural fit and sustainability), and what baseline should we compare against (e.g., a single-agent GPT-4 planner or a non-KG multi-agent version)?

By tackling this research problem, the project seeks to produce both an innovative software artifact (the orchestrator system itself) and empirical findings on multi-agent orchestration strategies in a real-world domain. The problem is inherently interdisciplinary, combining techniques from AI, software engineering (microservices, API protocols), and domain expertise (tourism management in Sri Lanka). The next sections will outline the objectives that break down this problem and the methodology we will follow to solve it.

2. OBJECTIVES

2.1 Main Objective

To design and evaluate a knowledge-graph-first multi-agent orchestration framework for Sri Lanka's tourism domain that enables context-aware, protocol-agnostic coordination of specialized agents, ultimately improving the quality, explainability, and sustainability of automated travel itinerary planning.

2.2 Specific Objectives

Design Orchestrator Core with Dynamic Agent Lifecycle Management

- Develop an Orchestrator Core capable of parsing natural language queries into structured intents and decomposing them into executable task graphs.
- Implement lifecycle management for agents (dynamic spawning, retirement, retries, timeouts, circuit breakers, and hot/cold pools).
- Deliver sub-components: Intent Parser, Task Graph Generator, Agent Manager, and Execution Controller.

Enable Multi-Protocol Agent Communication and Benchmarking

- Build adapters for at least three communication modes:
    - MCP (Model Context Protocol) for structured tool invocation,
    - A2A for direct peer-to-peer messaging,
    - Blackboard for KG-first shared memory.
- Compare protocols against one another on efficiency, latency, resilience, and developer complexity.

Develop a Tourism-Domain Agent Pack with Canonical Schemas

- Implement domain agents for Hotels, Transport, Visa/Policy, and Attractions (Dining optional in later phase).
- Ensure all agents follow a minimal canonical schema (e.g., Offer, TripLeg, PolicyRule, Evidence) for consistent input/output interoperability.

Implement Human-in-the-Loop Transactional Workflow

- Design HITL checkpoints for booking confirmation, ensuring user oversight before executing payments.
- Implement rollback and provenance logging so failed bookings trigger safe re-planning.

3. METHODOLOGY

The methodology of this research is centered on designing and implementing the **Multi-Agent Orchestration and Domain Agents** component, which acts as the central control mechanism of the intelligent tourism system. The project integrates several subsystems—speech and text processing, knowledge graph reasoning, recommendation and sustainability services—but the orchestration layer is the element that binds all of these together into a cohesive framework.

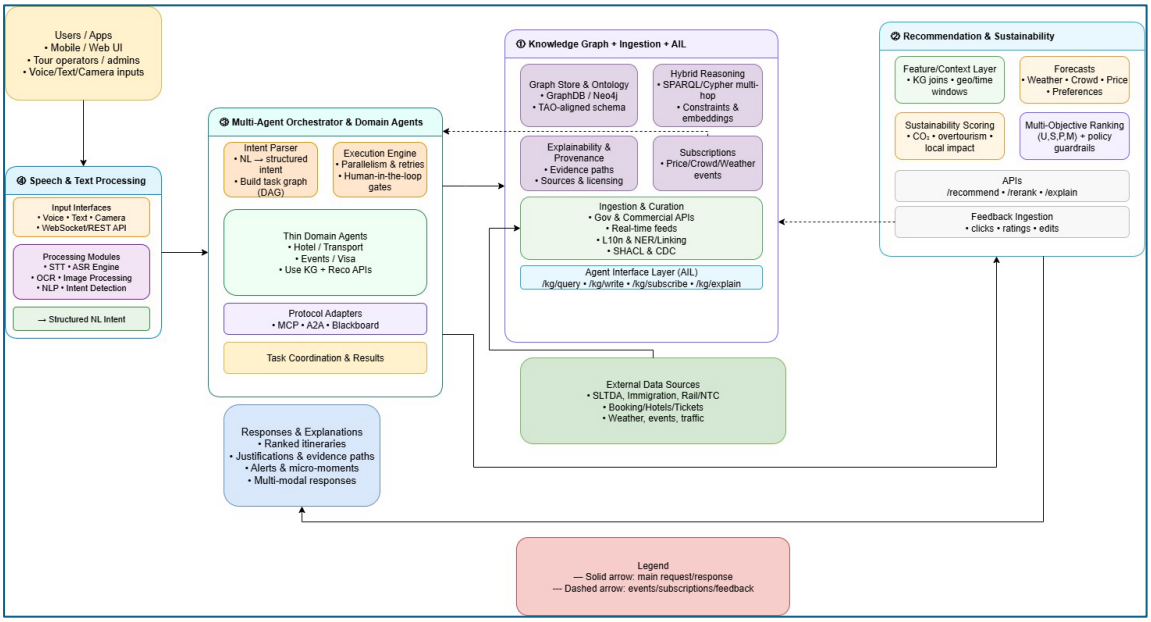

Figure 3.1:.Overall system Diagram

This diagram illustrates how user inputs flow through speech and text processing, are interpreted by the orchestrator, and are enriched by knowledge graph reasoning and recommendation services before returning as responses to the user. It provides readers with the context in which the orchestration and agent layer operates.

Following the placement of the overall system diagram, the methodology narrows down to the main focus of this research: the **Multi-Agent Orchestration and Domain Agents**.

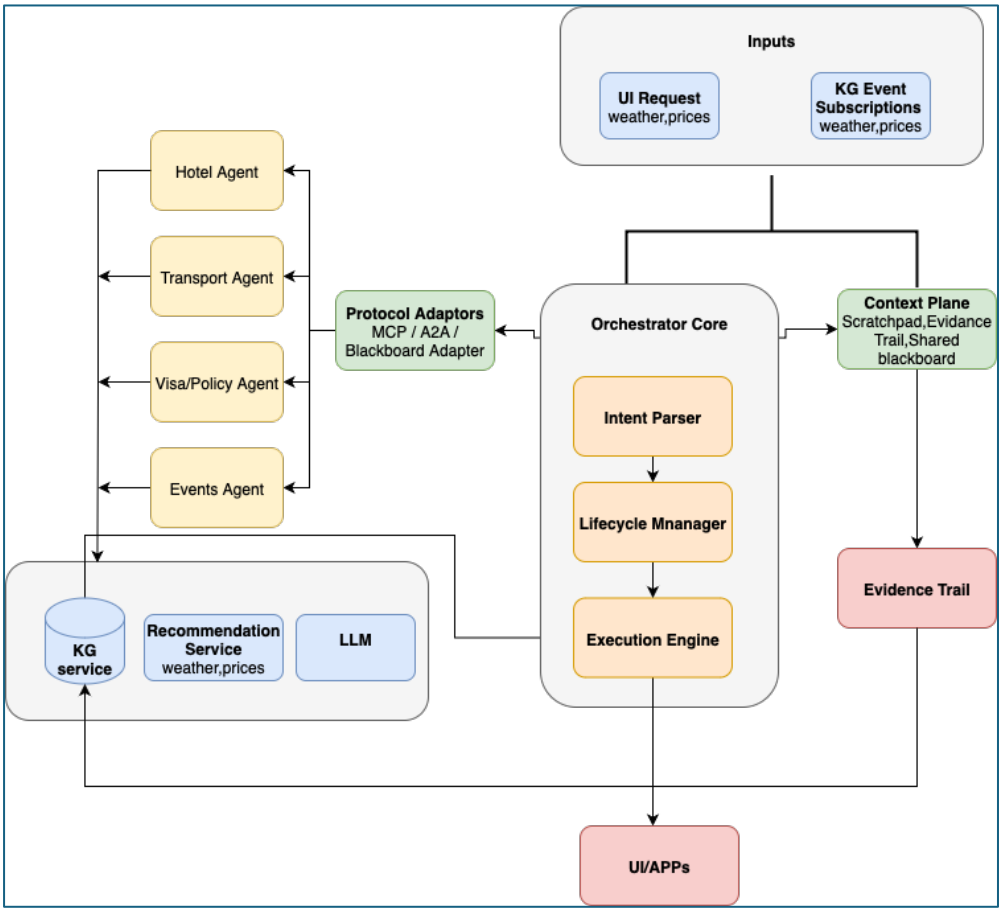

Figure 3.2 Component Diagram

This diagram zooms into the orchestrator core, protocol adapters, and domain agents, making it clear how the orchestration logic functions internally. Together, the two diagrams ensure that both the system-wide architecture and the focused component under study are well represented.

The orchestration component is designed around three major sub-functions: the orchestrator core, the domain agents, and the communication protocols that bind them together. The orchestrator core begins with an **intent parser**, which transforms user requests into structured intents by extracting goals, parameters, and constraints. These

intents are then managed by the **lifecycle manager**, which dynamically spawns and retires agents depending on the task at hand, while also handling retries, timeouts, and fault recovery to ensure robustness. Finally, the **execution engine** decomposes complex tourism queries into smaller, sequential sub-tasks and coordinates their completion by the relevant domain agents. This layered design ensures that the system can adapt flexibly to diverse and evolving user needs.

Each **domain agent** is developed as a specialized service that contributes unique expertise. The **Hotel Agent** handles accommodation offers and booking requests, while the **Transport Agent** is responsible for generating travel routes, timetables, and fares. The **Visa and Policy Agent** ensures that all planned itineraries comply with immigration rules, permits, and local restrictions, and the **Events Agent** incorporates cultural or seasonal activities into the itinerary. All agents communicate through a canonical schema, which standardizes outputs and prevents incompatibility when their results are merged by the orchestrator. This schema-driven approach also enables scalability, as new agents (for dining, local guides, or emergency assistance) can be added without redesigning the entire framework.

To guarantee effective collaboration among agents, the orchestrator integrates multiple communication protocols. The **Model Context Protocol (MCP)** supports structured tool invocation and API calls, enabling agents to interact with external data sources. The **Agent-to-Agent (A2A) protocol** facilitates direct peer-to-peer delegation of tasks, while the **Blackboard approach**, built on the shared knowledge graph, allows agents to store and retrieve contextual information with full evidence trails. This multi-protocol capability not only enhances flexibility but also allows benchmarking of each approach under identical tasks. For instance, planning a Colombo-to-Kandy itinerary during a Poya weekend can be executed once using MCP, again with A2A, and again using the Blackboard, with each run evaluated for latency, task success, and fault tolerance.

The implementation of this component will be carried out in phases. The first stage involves developing the orchestrator core and the initial set of domain agents. Once these are stable, the protocol adapters will be added and systematically tested. Benchmarking will then be conducted to measure the efficiency and reliability of the orchestrator under different modes of operation. Throughout this process, the orchestrator will continuously interact with the knowledge graph and recommendation layers, but its design ensures that its own performance and contributions to task success can be clearly isolated and studied.

In terms of resources, the component will rely on Python-based development with FastAPI for orchestrator services, Neo4j for the knowledge graph integration, Redis/Kafka for event handling, and Docker/Kubernetes for containerized deployment. External tourism datasets such as hotel booking platforms, transportation timetables, and policy datasets will be integrated via the agents, ensuring that the orchestrator is validated against realistic scenarios. The role of this component within the project team is led by me, with my work focusing exclusively on building, testing, and benchmarking the orchestration layer while other teammates handle the language service, knowledge graph, and sustainability modules.

The expected outcome is a fully operational **multi-agent orchestration framework** that can dynamically spawn and manage agents, coordinate them across multiple protocols, and deliver explainable and culturally appropriate itineraries. By the end of the development, the orchestrator is anticipated to achieve a high task success rate, low latency, and measurable gains in sustainability-aware planning. More importantly, it will demonstrate how domain-specific agents, when orchestrated effectively, can provide reliable and trustworthy decision support in real-world tourism applications.

4.PROJECT REQUIREMENTS

In this section, we outline the user requirements, system requirements, functional requirements, and non-functional requirements of the proposed system,

4.1 Functional Requirements

The system shall fulfill the following functional requirements (FR):

1. **FR1: Natural Language Query Understanding** – The system shall accept user queries in natural language related to travel planning (e.g., "Plan a 1-week trip to Sri Lanka including beach and cultural sites") and correctly interpret the user's intent (trip duration, interests, constraints). This entails extracting key parameters such as destinations, dates, preferences, and budget from the query. (Measured by how accurately the Intent Parser maps queries to tasks.)

2. **FR2: Task Decomposition and Agent Invocation** – The orchestrator shall decompose the planning problem into sub-tasks and assign these to the appropriate domain agents. For example, for any given trip planning request, it should spawn/search agents to handle accommodation, transport, attractions, and policy checking tasks at minimum. It should manage execution order based on task dependencies (e.g., get flight info before finalizing itinerary).

3. **FR3: Multi-Protocol Communication** – The orchestrator and agents shall communicate using at least two distinct protocols. Specifically: (a) via shared context (KG) where agents can read/write without direct calls, and (b) via direct messaging (HTTP or MCP). For each protocol mode, the information exchanged must remain consistent. For instance, if using MCP, the Hotel Agent'ssearch results should equal what it would provide via the blackboard method. We should be able to configure the system to use one mode or another for testing.

4. **FR4: Domain-Specific Agent Functions** – Each domain agent shall perform its specific functions as follows: Hotel Agent: Given a location and dates (and possibly budget or category), it returns a list of at least 3 suitable accommodation options with key details (price, rating, location). It also can handle a booking request for a selected hotel (simulated if not actually integrated).Transport Agent: Given origin, destination, and date or time window, it provides possible transport options (train, bus, car, etc.) including travel time and cost for each. If multiple legs are needed (e.g., connect flights or transfers), it should identify those as well.Attractions Agent: Given a location or region and a time slot (half-day/day) plus user interests, it recommends places or activities to fill that slot, including estimated durations and any scheduling constraints (like open hours). If given multiple days for a city, it should propose an itinerary schedule. Visa/Policy Agent: Given user nationality and trip plan (destinations, durations), it outputs any visa requirements or travel restrictions. Also, if any planned activity violates a known policy (like closed day or permit requirement), it flags it.

5. **FR5: Plan Synthesis** – The orchestrator shall compile outputs from various agents into a coherent trip plan. This includes sequencing days, assigning activities to days, incorporating travel times between locations, and ensuring the plan is feasible (no overlapping events, accounts for transit durations). The final plan should be human-readable (e.g., a day-by-day itinerary with narrative or bullet points) as well as machine-structured (for evaluation or modification).

6. **FR6: Explainability and Evidence** – For each major recommendation (hotel choice, route, attraction), the system shall provide an explanation or evidence source accessible to the user. This could be a reference (e.g., "Hotel recommended based on 4.5 rating on Booking.com") or reasoning ("Included Temple of Tooth

because you indicated interest in cultural sites"). The user should be able to request "why" and the system will answer based on evidence in the KG [5] .

7. **FR7: Human Approval & Booking Execution** – The system must obtain user confirmation of the finalized itinerary and specific bookings. Only after user approval will it proceed to "book" the itinerary elements. It should present a summary of items to be booked (with costs) for the user to review. Upon confirmation, it will mark those as booked in the system (and ideally provide confirmation details). If any booking fails, system will roll back as described or prompt user to resolve the issue. The system should clearly communicate booking success or failure.

These functional requirements ensure the system's features align with user expectations for a smart travel planning assistant and cover the collaborative and technical facets introduced by our approach.

4.2 Non-Functional Requirements

- **NFR1: Scalability & Modularity** – The architecture should allow adding new agent types or updating existing ones with minimal impact on the whole system. Adding a new domain agent (e.g., a Dining Agent) should ideally require writing that agent and plugging it in via the orchestrator config, without refactoring orchestrator core. The system should scale horizontally – e.g., if deployed on a cloud, one could run multiple orchestrators or agent instances if needed for load.

- **NFR2: Reliability & Fault Tolerance** – The system should be robust to individual component failures. If one agent crashes or a data source is unreachable, the orchestrator should not crash;it should handle the exception, maybe using a fallback or notifying the user.

- **NFR3: Security & Privacy** – The system may handle personal data (travel preferences, maybe names if integrated with booking). We must ensure secure handling: e.g., if the user's identity or payment details are to be used in booking (though we likely simulate this), we'd use secure channels (HTTPS for APIs) and not log sensitive info in plain text. Additionally, ensure agents cannot maliciously or accidentally tamper with each other's context beyond allowed operations (some access control on KG updates if needed). Also, if this were multi-user, the KG context separation per session must be enforced to avoid data leak between users.

- **NFR4: Maintainability** – The codebase should be organized (modular by agent and core, with clear interfaces) to allow future developers to maintain or extend. We will document the API for each agent and the orchestrator clearly. Using common languages (Python) and frameworks (FastAPI, standard Neo4j, etc.) also aids maintainability as new developers can pick it up more easily.

- **NFR5: Testability** – The system should be designed such that components can be tested independently (unit tests for agent logic, integration tests for orchestrator with dummy agent stubs, etc.). We should achieve good coverage on critical logic (particularly the orchestrator's decision flow and some critical agents' calculations). This ensures reliability and easier debugging.

These non-functional requirements ensure the delivered system is not only feature-complete but alsoefficient, reliable, and user-friendly, which are essential for real-world adoption and align with theresearch goal of demonstrating a robust approach.

4.3 System Requirements

- The system must integrate data from multiple sources including transport APIs, hotel booking platforms, weather APIs, and government data.
- The backend must support knowledge graph reasoning and AI-driven recommendation services.
- The system must ensure secure communication using HTTPS and strong authentication protocols.
- The system must provide APIs for third-party integration with external applications or services.

4.4 User Requirements

- The system must allow users to easily access tourism recommendations through a simple, multilingual interface.
- Users must be able to interact with the system in Sinhala, Tamil, and English, with smooth switching between languages.
- The system must provide accurate, up-to-date information on tourism services (transport, accommodation, events, weather).
- The interface must be accessible on mobile devices with minimal learning effort.

5.GANTT CHART

Task Name	Progress	Mar	Apr	May	Jun	July	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
Project Initiation																	
Study about research gaps	100%																
Deciding appropriate research area	100%																
Going through research papers	100%																
Finalizing a topic	100%																
Requirement Gathering & Analysis																	
Identifying functional requirements	100%																
Identifying non-functional requirements	100%																
Identifying data requirements	100%																
Identifying knowledge requirements	100%																
Design																	
Creating the Figma prototype	0%																
Development																	
Core Orchestrator Development	0%																
Protocol Integration	0%																
Domain Agent Development	0%																
Testing																	
Unit and API testing	0%																
Integrating Testing	0%																
Load testing	0%																
User Acceptance Testing	0%																
Deployment	0%																

Figure 5.1 Gantt Chart

5.1 Work Breakdown Structure

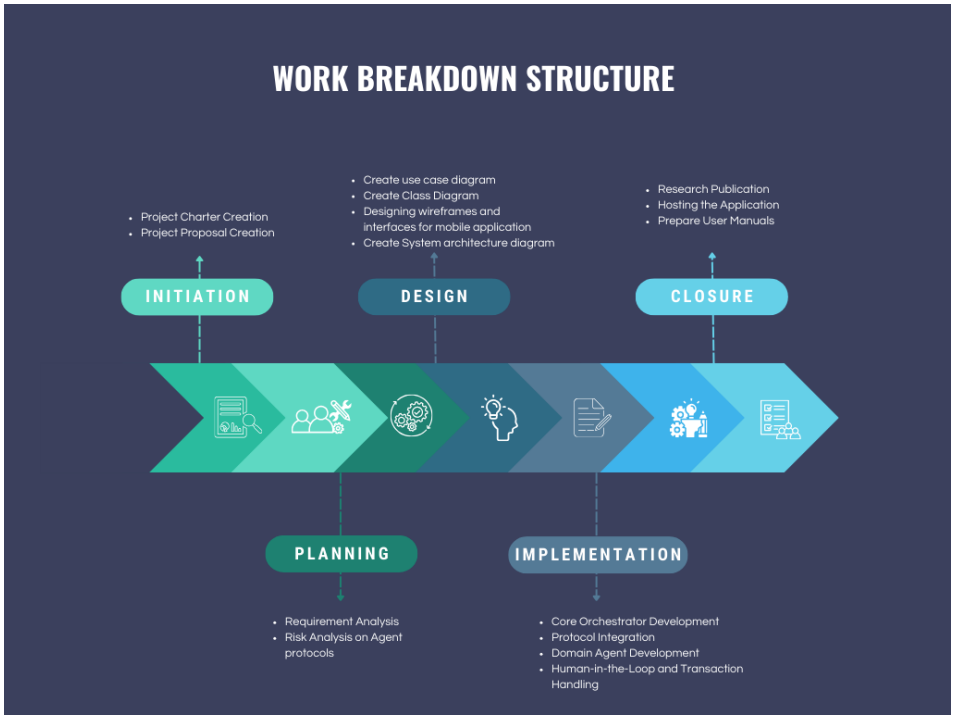

Figure 5.2 Work Breakdown Structure

6.PERSONAL AND FACILITIES

Facilitators:

- Dr. Lakmini Abeywardhena – Sri Lanka institute of Information Technology (SLIIT)
- Dr. Nathali Silva - Sri Lanka institute of Information Technology (SLIIT)

7. BUDGET AND BUDGET JUSTIFICATION

The proposed system has been planned with a lean but realistic budget, making use of free tiers where possible, while allocating essential costs for cloud services, APIs, and mobile deployment. The emphasis is on using **Azure cloud services** for backend hosting, storage, and speech capabilities, combined with **OpenAI API keys** for multi-agent reasoning and natural language tasks. In addition, a **Flutter-based mobile application** will be developed as the final deliverable, requiring costs for developer accounts on app stores. Hardware requirements such as microphones are also included to support data collection and testing. A buffer allocation has been added to account for unforeseen expenses during integration and field validation.

Description	Estimated Cost	Notes
Azure Cloud Services (VM hosting, Blob Storage, App Services, Speech-to-Text)	LKR 3000 / monthly (with free tier credits initially)	Backend hosting, data storage, STT/OCR services
OpenAI API Keys	LKR 3000	For embeddings, orchestration, reasoning tasks
Microphones for Voice Data Collection	LKR 10,000	For Sinhala–English code-mixed dataset collection
Mobile Testing Device	LKR 20,000	For app testing and evaluation

Miscellaneous / Buffer	LKR 10,000	Documentation, SD cards, adapters, printing

Table 7.1Budget Table

Total Budget = LKR 46,000

Budget Justification

Azure Cloud Services: Azure has been selected as the primary cloud provider due to its generous free tiers and reliable integration of storage, compute, and AI services. Virtual Machines and App Services will host the backend and APIs, while Blob Storage will hold datasets and graph exports. Azure Speech-to-Text will support Sinhala–English transcription at competitive rates, making it essential for the multilingual component.

OpenAI API Keys: A developer-tier plan will be used for embedding generation, agent orchestration, and conversational reasoning tasks. This ensures that the system can perform advanced semantic queries and multi-agent coordination effectively.

Flutter Mobile Application: The final deliverable will be a cross-platform mobile application built using Flutter, allowing deployment on both Android and iOS. Publishing costs include the Apple Developer Program and Google Play Developer registration. These are unavoidable costs to ensure the app can be distributed officially and maintained with regular updates.

Microphones & Mobile Device: A high-quality microphone (~LKR 10,000) is required for collecting Sinhala–English speech data to fine-tune STT components. Additionally, a mid-range Android device (~LKR 20,000) is budgeted for testing the mobile application in real-world conditions.

Miscellaneous Costs: A buffer of LKR10 000 is included for documentation, printing, SD cards, adapters, and other unforeseen requirements during development and evaluation.

7. REFERENCES

[1] Chen, A., Ge, X., Fu, Z., Xiao, Y., & Chen, J. (2024). Travelagent: An ai assistant for personalized travel planning. *arXiv preprint arXiv:2409.08069*.

[2] Singh, A., Madhogaria, R., Misra, A., & Elakiya, E. (2024). Automated Travel Planning Via Multi-Agent Systems and Real-Time Intelligence. *Available at SSRN 5089025*.

[3] Ehtesham, A., Singh, A., Gupta, G. K., & Kumar, S. (2025). A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp). *arXiv preprint arXiv:2505.02279*.

[4] Zhang, Y., Qiao, S., Zhang, J., Lin, T. H., Gao, C., & Li, Y. (2025). A survey of large language model empowered agents for recommendation and search: Towards next-generation information retrieval. *arXiv preprint arXiv:2503.05659*.

[5] N. Rohrseitz, "Knowledge Graphs for Multi-Agent Systems," *Medium*, Jul. 2025. [Online]. Available: https://medium.com/@nicolar/knowledge-graphs-for-multi-agent-systems-2025

[6] Song, S., Yang, C., Xu, L., Shang, H., Li, Z., & Chang, Y. (2024). TravelRAG: A Tourist Attraction Retrieval Framework Based on Multi-Layer Knowledge Graph. *ISPRS International Journal of Geo-Information*, *13*(11), 414.

[7] Liu, B., Ge, J., & Wang, J. (2025). Vaiage: A Multi-Agent Solution to Personalized Travel Planning. *arXiv preprint arXiv:2505.10922*.

[8] DAS, S., & Bagchi, M. (2025). An ontology-driven knowledge graph for tourism information management. *Open Research Europe*, *5*(1), 1.

[9] Singh, A., Madhogaria, R., Misra, A., & Elakiya, E. (2024). Automated Travel Planning Via Multi-Agent Systems and Real-Time Intelligence. *Available at SSRN 5089025*.

[10] Krishnan, N. (2025). Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications. *arXiv preprint arXiv:2504.21030*.

[11] Deng, J., Cvetkovic, A., Chung, P. K., Yankov, D., & Zhang, C. (2025). The User-Centric Geo-Experience: An LLM-Powered Framework for Enhanced Planning, Navigation, and Dynamic Adaptation. *arXiv preprint arXiv:2507.06993*.

[12] Liao, C. C., Liao, D., & Gadiraju, S. S. (2025). AgentMaster: A Multi-Agent Conversational Framework Using A2A and MCP Protocols for Multimodal Information Retrieval and Analysis. *arXiv preprint arXiv:2507.21105*.

[13] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... & Wang, C. (2024, August). Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.

[14] Aratchige, R. M., & Ilmini, W. M. K. S. (2025). Llms working in harmony: A survey on the technological aspects of building effective llm-based multi agent systems. *arXiv preprint arXiv:2504.01963*.

[15] Bei, Y., Zhang, W., Wang, S., Chen, W., Zhou, S., Chen, H., ... & Yu, P. S. (2025). Graphs Meet AI Agents: Taxonomy, Progress, and Future Opportunities. *arXiv preprint arXiv:2506.18019*.

[16] Besta, M., Paleari, L., Jiang, J. H. A., Gerstenberger, R., Wu, Y., Hannesson, J. G., ... & Hoefler, T. (2025). Affordable AI Assistants with Knowledge Graph of Thoughts. *arXiv preprint arXiv:2504.02670*.

[17] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., ... & Tang, J. (2023). Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*.

[18] Mo, G., Zhong, W., Chen, J., Chen, X., Lu, Y., Lin, H., ... & Sun, L. (2025). Livemcpbench: Can agents navigate an ocean of mcp tools?. *arXiv preprint arXiv:2508.01780*.